

# SIMATS ENGINEERING

## ASSESSMENT TOOL 1

**COURSE NAME: Artificial Intelligence**

**COURSE CODE: CSA17**

---

### COURSE OUTCOME COVERED

**C05:** *Design AI-based systems using PySpark, RDD operations, and intelligent agent architectures, using scalable systems and integrate components in distributed AI applications. (BTL 5)*

*Assessment Tool Weightage: 50%*

---

**QUESTIONS: 5**

Annexure A

**Duration : 1 Hour**  
**Total Marks:50**

### Design Task

---

#### **Code of Conduct:**

*I \_\_\_\_\_ (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

*I understand that any violation of academic integrity rules will result in disciplinary action.*

**Signature of the student:**

## Design Task

1. **Hybrid AI Recommendation Engine Design**  
Design a hybrid AI-based recommendation system that integrates content-based and collaborative filtering techniques using PySpark RDDs. Explain how distributed data processing can be used to handle large-scale user-item interactions, and describe the role of intelligent agents in adapting recommendations based on user behavior and feedback in real time.
2. **Smart Disaster Detection System**  
Develop a distributed AI system using PySpark and RDD operations to detect natural disasters (such as floods or earthquakes) from multi-source data streams (satellite images, sensors, and social media). Describe the system architecture, data fusion techniques, and how intelligent agents coordinate to provide early warnings and response strategies.
3. **AI-Based Fraud Detection Framework**  
Design a scalable fraud detection system using PySpark RDDs to process large volumes of financial transactions. Explain how anomaly detection algorithms can be implemented in a distributed manner and how intelligent agents collaborate to identify suspicious patterns, trigger alerts, and continuously improve detection accuracy.
4. **Autonomous Supply Chain Optimization System**  
Construct a distributed AI system using PySpark for optimizing supply chain operations (inventory, logistics, demand forecasting). Describe how RDD transformations support large-scale data processing and how multiple intelligent agents interact to make decentralized decisions for cost minimization and efficiency improvement.
5. **Personalized Learning Analytics Platform**  
Design an AI-driven educational platform that uses PySpark and RDDs to analyze student learning behavior and performance data. Explain how the system supports adaptive learning through intelligent agents, real-time feedback, and scalable analytics to personalize content delivery for thousands of learners.

# 1. Hybrid AI Recommendation Engine Design

## 1.1 Objective

A **hybrid recommendation engine** combines **content-based filtering** and **collaborative filtering** to recommend products, movies, courses, news articles, or other items to users.

The main objective is to overcome the limitations of using either technique alone:

- **Content-based filtering** recommends items similar to what a user has liked previously.

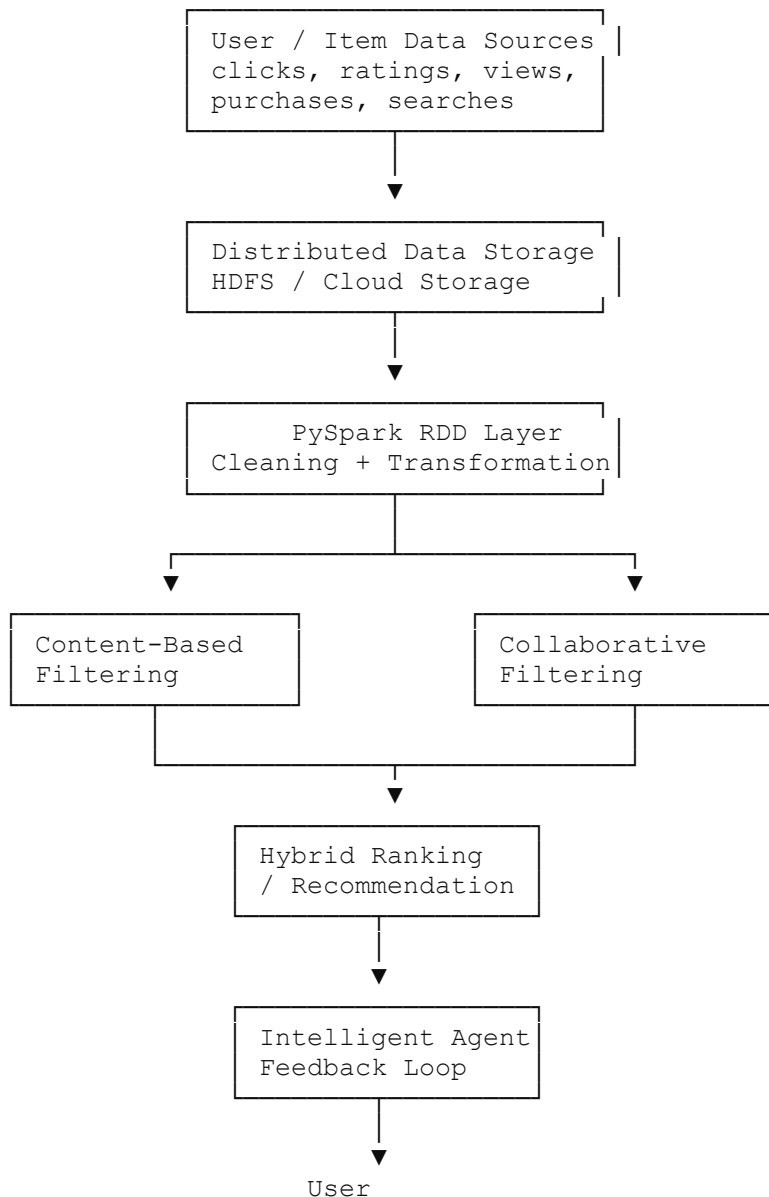
- **Collaborative filtering** recommends items based on the behavior of users with similar preferences.
- The **hybrid model** combines both sources of evidence to produce more accurate and diverse recommendations.
- **Intelligent agents** continuously observe user behavior and modify recommendations according to real-time feedback.

A typical large-scale system can process millions or billions of user-item interactions using **Apache Spark RDDs**.

---

## 1.2 System Architecture

A high-level architecture is:



## 1.3 Data Representation Using RDDs

Suppose the interaction data contains:

```
UserID, ItemID, Rating, Timestamp
U1, I10, 5, T1
U1, I20, 4, T2
U2, I10, 5, T3
U3, I30, 4, T4
```

It can be represented as:

```
ratings = sc.textFile("ratings.csv")
```

```
ratingsRDD = ratings.map(
    lambda x: x.split(",")
)
```

The data can then be converted to:

```
(user_id, item_id, rating)
```

For example:

```
userItemRDD = ratingsRDD.map(
    lambda x: (x[0], x[1], float(x[2]))
)
```

## 1.4 RDD Transformations

RDD transformations are useful because they are **distributed and lazy**.

Important transformations include:

`map()`

Used to transform individual records.

```
ratingsRDD.map(lambda x: x.split(","))
```

`filter()`

Used to remove invalid or irrelevant records.

```
ratingsRDD.filter(lambda x: x[2] != "")
```

`mapValues()`

Useful for transforming values while preserving keys.

```
userRatings.mapValues(lambda x: float(x))
```

`groupByKey()`

Groups records belonging to the same key.

```
userItemRDD.groupByKey()
```

`reduceByKey()`

Aggregates data efficiently.

```
ratings.reduceByKey(lambda a, b: a + b)
```

`join()`

Combines user, item, and metadata information.

```
userRDD.join(itemRDD)
```

## 1.5 Content-Based Filtering

Content-based filtering uses the characteristics of items.

For example, for a movie recommendation system, item attributes could include:

```
Movie ID
Genre
Director
Actors
Language
Year
Keywords
```

Each item can be represented as a feature vector:

Movie A → [Action=1, Comedy=0, Drama=1, SciFi=1]

Movie B → [Action=1, Comedy=0, Drama=0, SciFi=1]

The user's profile is generated from previously preferred items.

Similarity can be calculated using **cosine similarity**:

$$\text{Similarity}(A, B) = \frac{A \cdot B}{\|A\| \|B\|}$$

A high similarity score means that two items have similar characteristics.

### Example

If a user frequently watches:

- Science fiction
- Action
- Adventure

the system increases the user's preference weights for these features and recommends items containing similar features.

## 1.6 Collaborative Filtering

Collaborative filtering uses the behavior of many users.

Suppose:

| User | Movie A | Movie B | Movie C |
|------|---------|---------|---------|
| U1   | 5       | 4       | ?       |
| U2   | 5       | 4       | 5       |
| U3   | 1       | 2       | 1       |

U1 and U2 have similar preferences. Therefore, because U2 liked Movie C, Movie C can be recommended to U1.

Collaborative filtering can use:

- User-user similarity
- Item-item similarity
- Matrix factorization
- Implicit feedback

For a Spark RDD implementation, interactions can be represented as:

(user\_id, item\_id, rating)

Then users or items can be grouped and compared in parallel.

## 1.7 Hybrid Recommendation

The two scores can be combined:

$$\text{HybridScore} = \alpha \text{ContentScore} + (1 - \alpha) \text{CollaborativeScore}$$

where:

- ContentScore = similarity based on item features
- CollaborativeScore = similarity based on user behavior
- $\alpha$  = weighting factor

For example:

Content score = 0.80  
 Collaborative score = 0.70  
 $\alpha$  = 0.40

Then:

$$\text{HybridScore} = 0.4(0.80) + 0.6(0.70) = 0.74$$

The highest-scoring items are presented to the user.

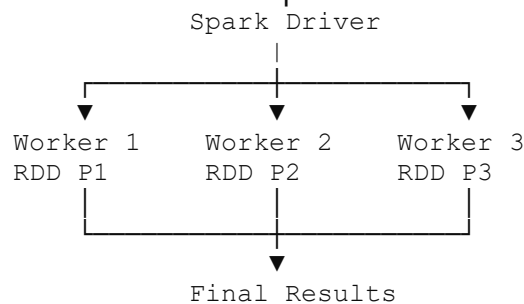
## 1.8 Distributed Processing

Suppose a website has:

- 50 million users
- 10 million products
- 5 billion interactions

Processing everything on a single machine is inefficient.

Spark distributes RDD partitions across worker nodes:



Operations such as map, filter, and reduceByKey can execute in parallel.

RDDs also provide **fault tolerance** through lineage. If a partition is lost, Spark can reconstruct it from the transformations that created it.

## 1.9 Intelligent Agents

Intelligent agents provide the adaptive component.

A recommendation agent observes:

- Clicks
- Purchases
- Ratings
- Search queries
- Time spent
- Skipped items
- Likes/dislikes
- Recently viewed items

The agent updates the user's preference model.

For example:

User watches:

Action → +2

Science Fiction → +3

Comedy → +1

Romance → -1

The agent adjusts recommendation weights accordingly.

Multiple agents can be used:

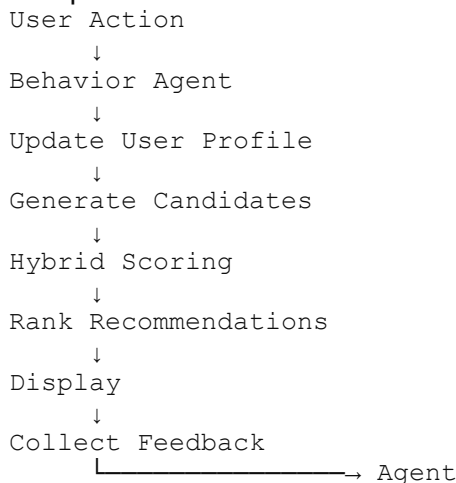
- **User Agent** – maintains individual user preferences.
- **Content Agent** – analyzes item characteristics.
- **Recommendation Agent** – generates candidate items.
- **Feedback Agent** – analyzes user reactions.
- **Ranking Agent** – orders recommendations.
- **Monitoring Agent** – monitors recommendation quality.

## 1.10 Real-Time Adaptation

Suppose a user suddenly starts watching cricket videos.

The intelligent agent detects this behavioral change and temporarily increases the weight of cricket-related content.

The process becomes:



This creates a continuous learning loop.

---

## 1.11 Advantages

- Handles very large datasets.
- Combines multiple recommendation strategies.
- Scales horizontally.
- Supports personalization.
- Can incorporate real-time feedback.
- RDDs provide distributed fault-tolerant processing.
- Intelligent agents enable adaptive behavior.

## 2. Smart Disaster Detection System

### 2.1 Objective

A **Smart Disaster Detection System** combines data from:

- Satellite imagery
- IoT sensors
- Weather stations
- Seismic sensors
- River-level sensors
- Social media
- Emergency reports

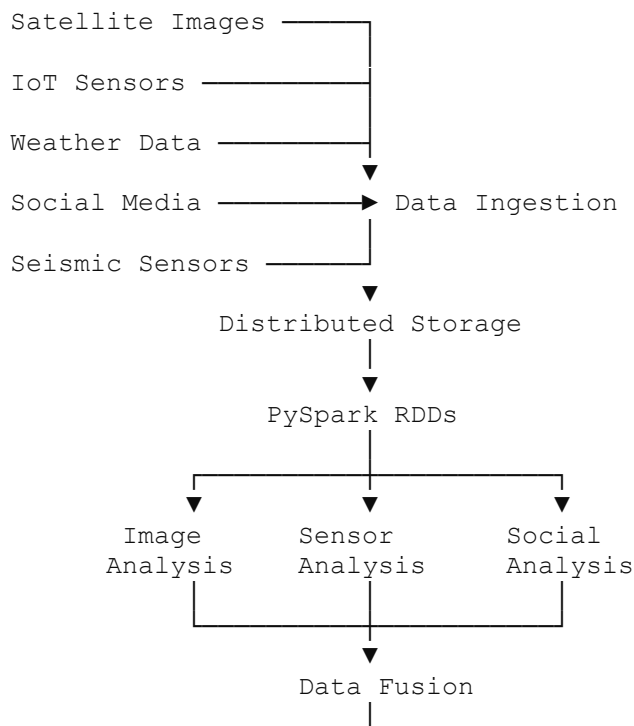
The objective is to detect disasters such as:

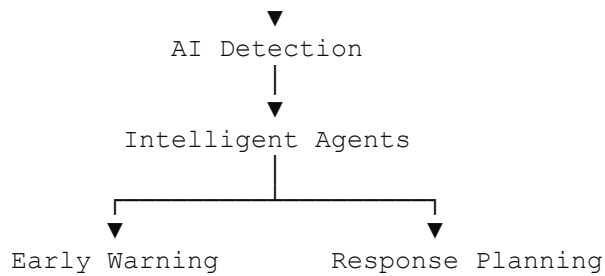
- Floods
- Earthquakes
- Landslides
- Cyclones
- Wildfires

as early as possible.

The system uses **PySpark RDDs** for large-scale distributed processing and intelligent agents for monitoring, data fusion, warning generation, and response coordination.

### 2.2 Architecture





## 2.3 Data Sources

### Satellite Images

Satellite data can provide:

- Water coverage
- Cloud patterns
- Vegetation changes
- Burned areas
- Terrain changes

Image processing can extract relevant features before they are represented in Spark data structures.

### Sensors

Sensors can continuously provide:

Sensor ID  
Latitude  
Longitude  
Timestamp  
Water Level  
Temperature  
Pressure  
Seismic Activity

### Social Media

Social media can provide early indications such as:

"Water entering our street"  
"Large earthquake just happened"  
"Bridge collapsed"

However, social media data should be treated as **unverified evidence** until corroborated by trusted sensors or authorities.

## 2.4 RDD Processing

Sensor records can be represented as:

```
sensorRDD = sc.textFile("sensor_data")
```

Then:

```
cleanRDD = sensorRDD.filter(  
    lambda x: x is not None  
)
```

Records can be mapped into:

```
(location, water_level)
```

Then grouped:

```
waterRDD.groupByKey()
```

The system can calculate average, maximum, or rate-of-change values using distributed operations.

For example:

```
maxWater = waterRDD.reduceByKey(  
    lambda a, b: max(a, b)  
)
```

## 2.5 Data Fusion

Data fusion means combining information from different sources.

For example:

River Sensor:  
Water level = Very High

Weather Data:  
Heavy rainfall predicted

Satellite:  
Large water-covered region detected

Social Media:  
Multiple flood reports

Individually, each source may be uncertain.

Together, they provide stronger evidence.

A fusion score can be represented as:

$$D = w_1S + w_2W + w_3I + w_4M$$

where:

- $S$  = sensor evidence
- $W$  = weather evidence
- $I$  = satellite/image evidence
- $M$  = social-media evidence
- $w_i$  = reliability weights

## 2.6 Example: Flood Detection

Suppose:

|                    |        |
|--------------------|--------|
| Sensor evidence    | = 0.90 |
| Satellite evidence | = 0.85 |
| Weather evidence   | = 0.80 |
| Social evidence    | = 0.75 |

With appropriate reliability weights, the fusion system may produce:

Flood Probability = 0.86

If:

Probability > 0.80

the system can generate a high-priority alert for human verification and emergency response.

For safety-critical systems, automated alerts should use carefully validated thresholds and authoritative emergency procedures rather than relying solely on an AI prediction.

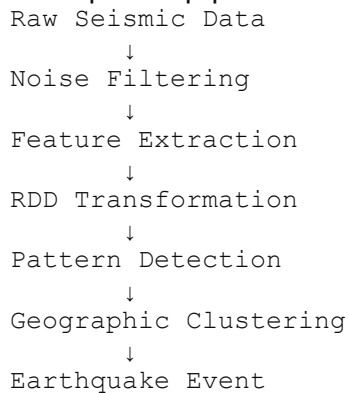
## 2.7 Earthquake Detection

Seismic sensors generate signals continuously.

RDD processing can:

1. Partition seismic measurements.
2. Remove noise.
3. Detect abnormal amplitude/frequency patterns.
4. Group measurements geographically.
5. Identify correlated events.
6. Estimate event location and severity.

A simplified pipeline:



## 2.8 Intelligent Agents

Several intelligent agents can collaborate.

### **Sensor Agent**

Monitors IoT and environmental sensors.

### **Satellite Agent**

Analyzes incoming satellite observations.

### **Social Monitoring Agent**

Identifies disaster-related reports and trends.

### **Fusion Agent**

Combines evidence from multiple sources.

### **Prediction Agent**

Estimates disaster probability and severity.

### **Alert Agent**

Generates warnings according to validated thresholds.

### **Response Agent**

Recommends actions such as:

- Evacuation routes
- Emergency resource allocation
- Medical support
- Rescue team deployment

## Coordination Agent

Ensures that information from different agents is consistent and prioritized.

## 2.9 Early Warning

The system can classify warnings:

Level 1 → Normal

Level 2 → Monitoring

Level 3 → Potential Hazard

Level 4 → High Risk

Level 5 → Emergency

The intelligent agent continuously updates the state.

For example:

10:00 → Normal

10:10 → Increased rainfall

10:20 → River level rising

10:30 → Satellite confirms flooding

10:35 → High-risk warning

## 2.10 Advantages

- Combines heterogeneous information.
- Processes high-volume sensor streams.
- Detects spatial and temporal patterns.
- Provides faster situational awareness.
- Supports scalable disaster monitoring.
- Intelligent agents enable coordination between specialized detection modules.

# 3. AI-Based Fraud Detection Framework

## 3.1 Objective

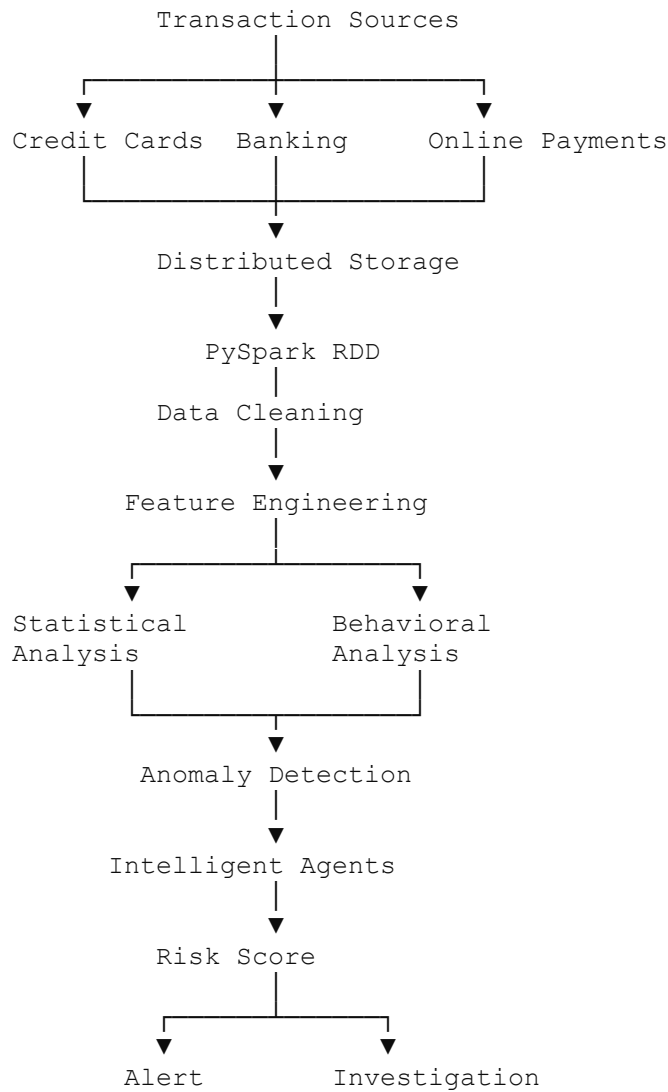
A fraud detection system identifies suspicious financial transactions from massive transaction datasets.

Typical examples include:

- Credit-card fraud
- Online-payment fraud
- Account takeover
- Money laundering indicators
- Fake transactions
- Unusual merchant behavior

The system must process transactions at very large scale while minimizing false positives.

## 3.2 Architecture



## 3.3 Transaction Data

A transaction may contain:

Transaction ID  
User ID  
Amount  
Merchant ID  
Location  
Timestamp  
Payment Method  
Device ID  
IP Address

An RDD could contain:

```
transactions = sc.textFile("transactions.csv")
```

The records can be transformed into:

```
(user_id, amount, location, timestamp)
```

## 3.4 Data Preprocessing

Fraud detection requires high-quality data.

Typical operations:

- Remove corrupted records.
- Handle missing values.
- Normalize numerical values.
- Remove duplicate transactions.
- Convert timestamps.
- Validate transaction amounts.
- Generate behavioral features.

For example:

```
clean = transactions.filter(
    lambda x: valid_transaction(x)
)
```

## 3.5 Feature Engineering

Useful features include:

### Transaction features

- Transaction amount
- Merchant category
- Time of transaction
- Geographic location

### Behavioral features

- Average spending
- Number of transactions per hour
- Number of merchants visited
- Typical transaction location

### Velocity features

For example:

10 transactions within 2 minutes

may be suspicious depending on the user's normal behavior.

### Geographic features

For example:

Transaction A → Chennai

Transaction B → London

10 minutes later

could require investigation, although legitimate explanations may exist.

## 3.6 Anomaly Detection

Anomaly detection identifies observations that differ significantly from normal behavior.

One simple approach is a statistical z-score:

$$z = \frac{x - \mu}{\sigma}$$

where:

- $x$  = current transaction value
- $\mu$  = historical mean
- $\sigma$  = standard deviation

Large deviations can receive higher risk scores.

However, production systems generally combine multiple signals rather than relying on one statistical rule.

## 3.7 Distributed Anomaly Detection

Suppose billions of transactions exist.

RDD partitions can independently calculate intermediate statistics:

```
Partition 1 → statistics
Partition 2 → statistics
Partition 3 → statistics
Partition 4 → statistics
```

The results can then be aggregated.

For example:

```
amountRDD.reduce(
    lambda a, b: a + b
)
```

For user-level behavior:

```
userTransactions = transactions.map(
    lambda x: (x.user_id, x)
)
```

```
grouped = userTransactions.groupByKey()
```

The system can then build behavioral profiles for each user.

## 3.8 Risk Scoring

A combined risk score can be calculated as:

$$Risk = w_1A + w_2V + w_3L + w_4B + w_5D$$

where:

- $A$  = amount anomaly
- $V$  = transaction velocity anomaly
- $L$  = location anomaly
- $B$  = behavioral anomaly
- $D$  = device anomaly

Example:

```
Amount anomaly      = 0.8
Location anomaly     = 0.9
Behavior anomaly     = 0.7
Device anomaly       = 0.6
```

```
Overall risk        = 0.79
```

Transactions above a validated threshold can be sent to an investigation or review workflow.

## 3.9 Intelligent Agents

### Transaction Monitoring Agent

Examines transactions as they arrive.

### User Behavior Agent

Maintains a dynamic profile of normal user behavior.

### Pattern Detection Agent

Looks for suspicious sequences.

### Network Agent

Examines relationships between:

- Accounts
- Devices
- Merchants
- IP addresses

### Risk Agent

Combines evidence and calculates risk.

### Alert Agent

Generates alerts for transactions requiring investigation.

### Learning Agent

Uses confirmed investigation outcomes to improve future detection.

## 3.10 Agent Collaboration

Consider:

```
Transaction
  ↓
Monitoring Agent
  ↓
Behavior Agent → unusual amount
  ↓
Location Agent → unusual location
  ↓
Device Agent → new device
  ↓
Risk Agent
  ↓
High Risk
  ↓
Alert Agent
```

The agents do not necessarily need to make independent final decisions. A centralized risk layer can combine their outputs while preserving auditability and consistent policy enforcement.

## 3.11 Continuous Learning

Fraud patterns change over time.

A previously unknown fraud pattern might initially evade detection.

After investigators label transactions:

Confirmed Fraud



Learning Agent



Update Features/Model



New Detection Rules



Future Transactions

This makes the system adaptive.

## 3.12 Advantages

- Handles massive transaction volumes.
- Distributed computation improves scalability.
- Detects behavioral anomalies.
- Supports real-time or near-real-time risk scoring.
- Multiple agents provide specialized analysis.
- Feedback improves future detection.

# 4. Autonomous Supply Chain Optimization System

## 4.1 Objective

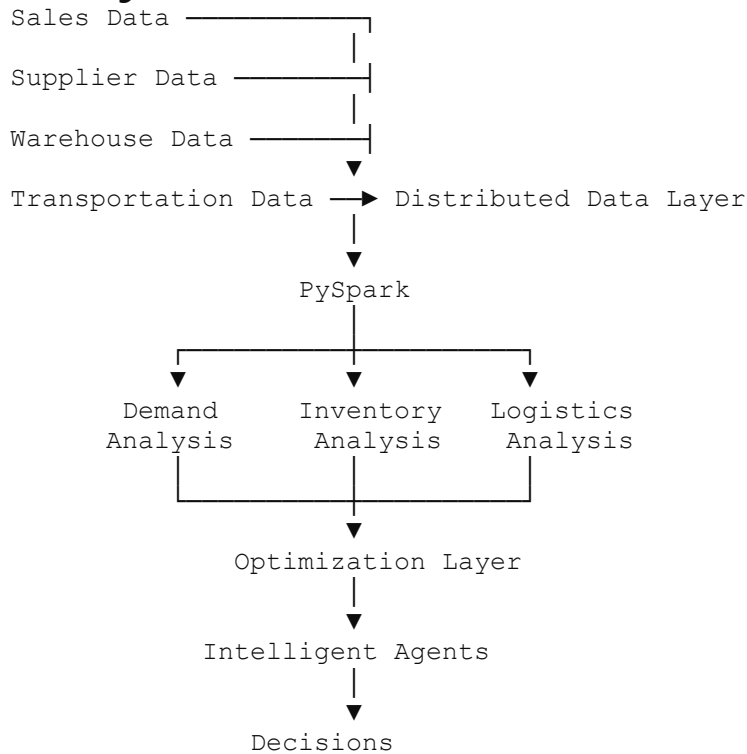
The objective is to optimize:

- Inventory
- Procurement
- Warehousing
- Transportation
- Demand forecasting
- Supplier selection
- Delivery scheduling

while minimizing:

- Cost
- Stockouts
- Overstocking
- Transportation time
- Waste

## 4.2 System Architecture



## 4.3 Data Processing Using RDDs

Historical sales can be represented as:

(ProductID, Date, Quantity, Location)

Create an RDD:

```
salesRDD = sc.textFile("sales.csv")
```

Transform:

```
productSales = salesRDD.map(  
    lambda x: (get_product(x), get_quantity(x))  
)
```

Aggregate:

```
totalSales = productSales.reduceByKey(  
    lambda a, b: a + b  
)
```

This allows sales for millions of products to be aggregated in parallel.

## 4.4 Demand Forecasting

Historical demand can be processed to identify:

- Daily demand
- Weekly demand
- Seasonal trends
- Product popularity
- Regional demand

For example:

Product A

Week 1 → 100

Week 2 → 120

Week 3 → 130

Week 4 → 150

The forecasting module predicts future demand.

The prediction can then influence inventory decisions.

## 4.5 Inventory Optimization

The system continuously calculates:

Current inventory

+

Expected incoming inventory

-

Forecast demand

=

Expected inventory

If expected inventory falls below a safety threshold, a replenishment agent can recommend ordering more.

A simplified reorder point is:

$$ROP = Demand_{leadtime} + SafetyStock$$

## 4.6 Logistics Optimization

Transportation data may include:

Source

Destination

Distance

Vehicle

Capacity

Fuel Cost

Delivery Time

RDD transformations can group shipments by:

- Region
- Warehouse
- Delivery date
- Vehicle
- Customer

This supports route and shipment optimization.

## 4.7 Intelligent Agents

### Demand Agent

Forecasts future product demand.

### Inventory Agent

Maintains stock levels.

### Procurement Agent

Determines when and how much to purchase.

### Supplier Agent

Evaluates suppliers using:

- Price
- Reliability
- Lead time
- Quality

### **Logistics Agent**

Optimizes transportation.

### **Warehouse Agent**

Optimizes warehouse allocation.

### **Coordination Agent**

Resolves conflicts between agents.

## **4.8 Decentralized Decision Making**

Consider three warehouses:

Warehouse A → Overstocked

Warehouse B → Low inventory

Warehouse C → Normal

The warehouse agents exchange information.

The system may decide:

Transfer 500 units:

A → B

instead of purchasing new inventory.

This reduces cost.

## **4.9 Cost Function**

The optimization objective can be represented as:

$$TotalCost = InventoryCost + TransportationCost + OrderingCost + StockoutCost$$

The intelligent agents attempt to minimize this function while meeting service-level requirements.

## **4.10 Example**

Suppose:

Warehouse A:

Inventory = 5,000 units

Warehouse B:

Inventory = 500 units

Forecast demand at B:

1,000 units

Transportation cost A → B:

Low

The inventory agent detects a potential shortage.

The logistics agent determines that moving 600 units from A to B is cheaper than placing an urgent supplier order.

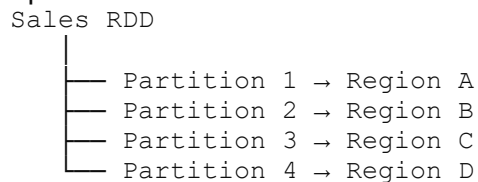
The coordination agent approves the plan subject to business constraints.

## 4.11 Distributed Processing

Large organizations can have:

- Millions of sales records
- Thousands of suppliers
- Thousands of warehouses
- Millions of shipments

Spark divides the workload across nodes:



Operations such as map, filter, reduceByKey, groupByKey, and join allow large-scale analysis.

## 4.12 Advantages

- Reduces inventory costs.
- Reduces stockouts.
- Improves demand forecasting.
- Optimizes logistics.
- Supports decentralized decision making.
- Scales to large supply-chain datasets.
- Intelligent agents continuously adapt decisions to changing conditions.

# 5. Personalized Learning Analytics Platform

## 5.1 Objective

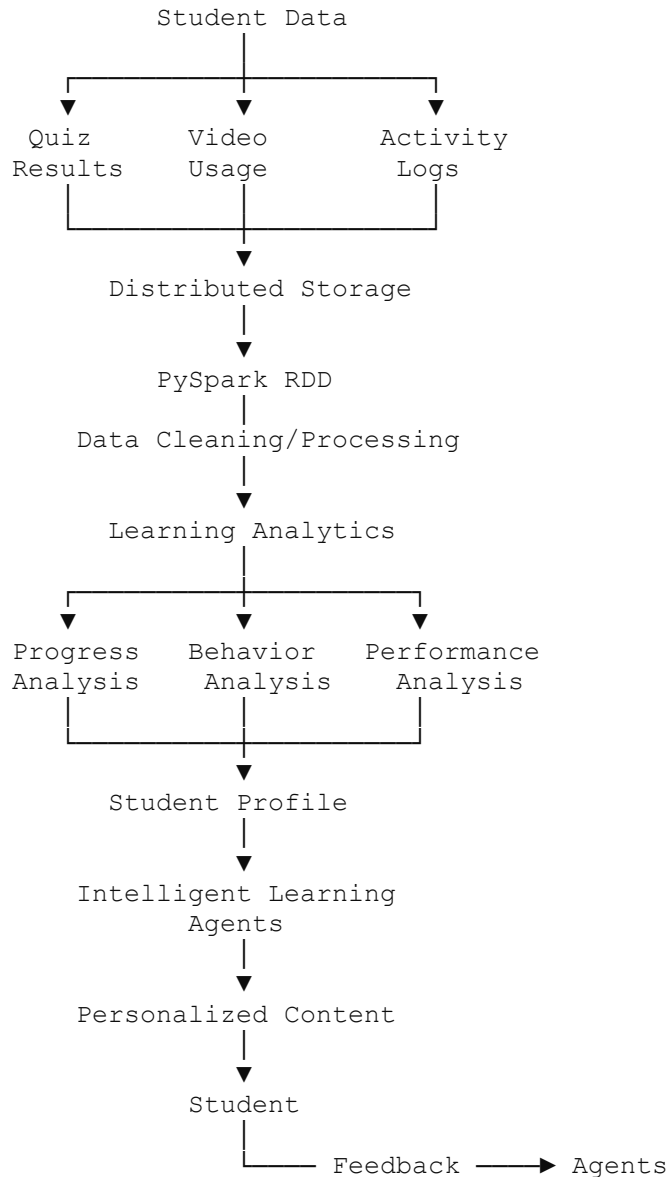
A personalized learning platform analyzes the behavior and performance of thousands or millions of learners.

It can personalize:

- Courses
- Videos
- Exercises
- Quizzes
- Reading materials
- Difficulty levels
- Revision activities

The system uses **PySpark RDDs** for scalable analytics and intelligent agents for adaptive learning.

## 5.2 Architecture



## 5.3 Student Data

The system can collect:

Student ID  
Course ID  
Lesson ID  
Quiz Score  
Attempt Count  
Time Spent  
Completion Rate  
Video Watch Percentage  
Questions Attempted  
Timestamp

For example:

```
learningRDD = sc.textFile("learning_activity.csv")
```

Records can be transformed into:

```
(student_id, activity)
```

## 5.4 Distributed Analytics

Suppose there are 10 million students.

Each student's activity can be distributed across Spark partitions.

For example:

```
studentRDD = learningRDD.map(  
    lambda x: (get_student_id(x), parse_activity(x))  
)
```

Student records can be grouped:

```
studentGroups = studentRDD.groupByKey()
```

The platform can calculate:

- Average score
- Completion rate
- Number of attempts
- Time spent
- Weak topics
- Strong topics

## 5.5 Performance Analysis

Consider:

Mathematics → 85%

Physics → 72%

Programming → 95%

Statistics → 55%

The system identifies Statistics as a weak area.

The student's learning profile becomes:

Strong:

Programming

Mathematics

Moderate:

Physics

Needs improvement:

Statistics

The recommendation engine can then provide additional Statistics material.

## 5.6 Adaptive Learning

The learning system can change difficulty dynamically.

For example:

Student answers easy questions correctly

↓

Agent increases difficulty

Student repeatedly fails advanced questions

↓

Agent reduces difficulty

↓

Provides prerequisite material

↓  
Student practices  
↓  
Agent evaluates improvement  
This is more effective than giving every student identical content.

## 5.7 Intelligent Agents

### Student Profile Agent

Maintains an evolving learner profile.

### Content Agent

Analyzes educational resources and their difficulty.

### Assessment Agent

Analyzes quiz and assignment results.

### Recommendation Agent

Selects the next learning resource.

### Feedback Agent

Provides immediate feedback.

### Motivation Agent

Can recommend reminders, revision sessions, or learning goals.

### Instructor Agent

Provides teachers with aggregated information about class performance.

## 5.8 Real-Time Feedback

Suppose a learner answers five questions incorrectly on the same topic.

The system detects the pattern:

Incorrect answers  
↓  
Assessment Agent  
↓  
Weak concept detected  
↓  
Student Profile Agent  
↓  
Recommendation Agent  
↓  
Prerequisite lesson  
↓  
Practice questions

After the student completes the material, the system evaluates the improvement.

## 5.9 Learning Recommendation Score

A recommendation score could combine:

$$\text{Score} = w_1 \text{KnowledgeGap} + w_2 \text{DifficultyFit} + w_3 \text{Interest} + w_4 \text{RecentPerformance} + w_5 \text{ContentQuality}$$

The platform ranks available learning resources using this score.

For example:

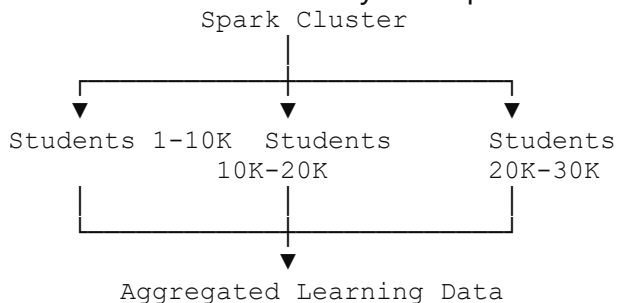
Resource A  $\rightarrow$  0.92

Resource B → 0.81  
Resource C → 0.67  
Resource D → 0.55

Resource A would be recommended first.

## 5.10 Scaling to Thousands of Learners

RDDs allow student activity to be processed in parallel:



This makes it possible to analyze large volumes of educational events.

## 5.11 Example End-to-End Scenario

A student is learning Python.

Their activity is:

Lesson 1 → completed  
Quiz 1 → 90%  
Lesson 2 → completed  
Quiz 2 → 85%  
Lesson 3 → completed  
Quiz 3 → 45%

The assessment agent detects a performance decline.

The student profile agent identifies that Lesson 3's concepts are weak.

The recommendation agent therefore provides:

1. Review Lesson 3
2. Watch an easier explanation
3. Complete prerequisite exercises
4. Attempt a short practice quiz
5. Retry the original assessment

If the student improves:

45% → 75% → 88%

the agent increases the student's estimated mastery and moves them toward more advanced material.

## 5.12 Privacy and Security

A learning analytics system should also protect student information.

Important measures include:

- Access control
- Encryption
- Data minimization
- Secure authentication
- Anonymization/pseudonymization where appropriate

- Audit logging
- Appropriate retention policies

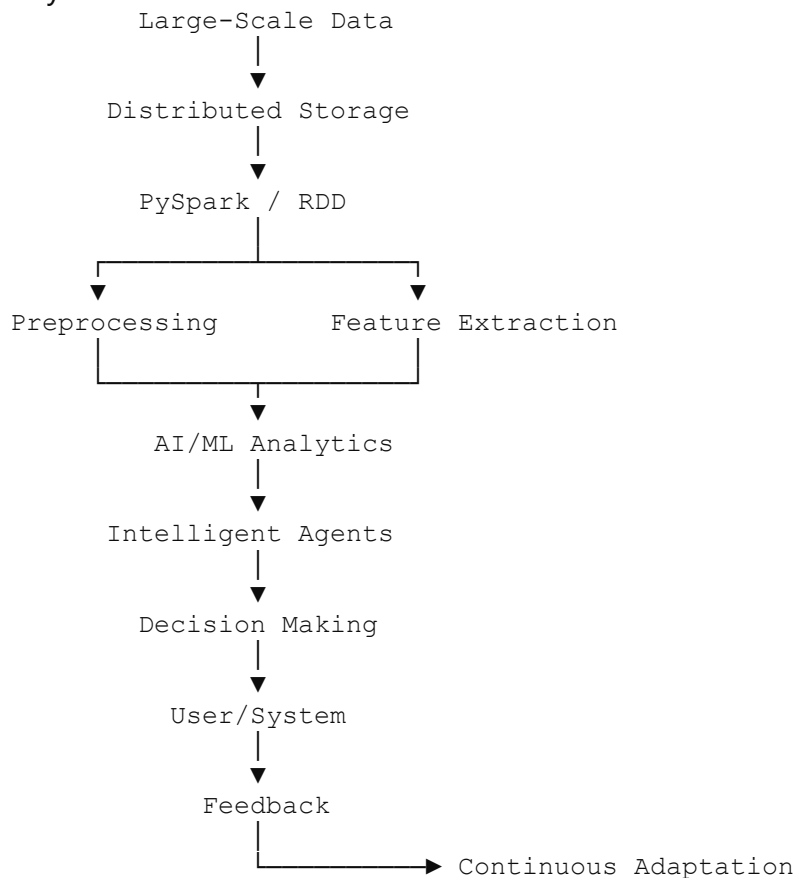
The system should avoid using sensitive student information unless there is a legitimate educational purpose and appropriate authorization.

## Overall Comparison

| System                | Main Data                        | PySpark/RDD Role                                | Intelligent Agents                          |
|-----------------------|----------------------------------|-------------------------------------------------|---------------------------------------------|
| Recommendation Engine | User-item interactions           | Distributed similarity and aggregation          | Personalization and feedback                |
| Disaster Detection    | Sensors, satellite, social media | Distributed event and feature processing        | Detection, fusion, warning, response        |
| Fraud Detection       | Financial transactions           | Large-scale anomaly analysis                    | Risk analysis and alert coordination        |
| Supply Chain          | Sales, inventory, logistics      | Distributed aggregation and optimization inputs | Procurement, inventory, logistics decisions |
| Learning Analytics    | Student activity                 | Distributed learner-performance analysis        | Adaptive learning and feedback              |

## Common Design Pattern

All five systems follow a similar distributed-AI architecture:



## Why PySpark RDDs are useful

RDDs are particularly useful for these systems because they provide:

- **Distributed processing** — data can be processed across many worker nodes.
- **Fault tolerance** — lost partitions can be reconstructed through lineage.
- **Parallel transformations** — map, filter, flatMap, groupByKey, reduceByKey, and join support large-scale processing.
- **In-memory computation** — repeatedly used datasets can be cached.
- **Scalability** — additional machines can be added as data volume grows.
- **Data locality** — computation can be performed close to the data where the execution environment supports it.

## Role of Intelligent Agents

The intelligent-agent layer adds **autonomy and adaptation** to the distributed data-processing layer.

A useful conceptual model is:

*Observe → Analyze → Decide → Act → ReceiveFeedback → Learn*

Thus, **PySpark/RDDs provide the scalable computational infrastructure**, while **AI models provide prediction or detection**, and **intelligent agents provide adaptive decision-making and coordination**.

For a real production implementation, especially in disaster response, financial fraud, and education, the agent decisions should also include **human oversight, explainability, monitoring, security, and explicit policy constraints** rather than allowing an AI system to make unrestricted autonomous decisions

## RUBRICS FOR CALCULATION

| Evaluation Criteria | Problem Understanding & Requirement Analysis | System Architecture Design | Use of PySpark & RDD Operations | Intelligent Agent Integration | Scalability & Distributed Computing Design | Data Flow & Processing Pipeline | Innovation & Creativity | Clarity, Presentation & Documentation |
|---------------------|----------------------------------------------|----------------------------|---------------------------------|-------------------------------|--------------------------------------------|---------------------------------|-------------------------|---------------------------------------|
| Weightage           | 10%                                          | 20%                        | 15%                             | 15%                           | 10%                                        | 10%                             | 10%                     | 10%                                   |

| Criteria                                     | Excellent (5)                                                  | Good (4)                      | Average (3)                | Poor (2)             |
|----------------------------------------------|----------------------------------------------------------------|-------------------------------|----------------------------|----------------------|
| Problem Understanding & Requirement Analysis | Clearly defines problem, objectives, and constraints           | Mostly clear with minor gaps  | Basic understanding        | Unclear or incorrect |
| System Architecture Design                   | Complete, scalable, well-structured architecture with diagrams | Good design with minor issues | Basic design, lacks detail | Poor/incomplete      |
| Use of PySpark & RDD Operations              | Efficient and correct use of RDD transformations/actions       | Mostly correct usage          | Limited or partial use     | Incorrect/missing    |
| Intelligent Agent Integration                | Strong integration with clear agent roles and logic            | Good integration              | Basic integration          | Poor/no integration  |
| Scalability & Distributed Computing Design   | Clearly addresses scalability, fault tolerance, parallelism    | Covers most aspects           | Limited consideration      | Not addressed        |

|                                       |                                                      |                 |                    |                   |
|---------------------------------------|------------------------------------------------------|-----------------|--------------------|-------------------|
| Data Flow & Processing Pipeline       | Clear, logical, and well-defined data flow           | Mostly clear    | Some gaps          | Poorly defined    |
| Innovation & Creativity               | Highly innovative, realistic, and impactful solution | Some innovation | Basic idea         | No originality    |
| Clarity, Presentation & Documentation | Very clear, neat diagrams, well-organized report     | Mostly clear    | Somewhat organized | Poor presentation |